Assignment 9: Advanced SQL Queries Using Joins and Subqueries

Database: MySQL | Project: Airline Reservation System

1. Objective

Use joins and subqueries to generate reports involving multiple tables, related records and summary statistics.

2. Report 1 — booking details using multiple joins

```
SELECT p.full_name AS passenger_name,
       f.flight_number, f.departure_airport,
       f.arrival_airport, a.model AS aircraft,
       b.seat_number, b.travel_class,
       pay.amount, pay.payment_status
FROM Bookings b
JOIN Passengers p ON b.passenger_id=p.passenger_id
JOIN Flights f ON b.flight_id=f.flight_id
JOIN Aircraft a ON f.aircraft_id=a.aircraft_id
LEFT JOIN Payments pay ON b.booking_id=pay.booking_id
ORDER BY p.passenger_id, f.flight_id;
```

passenger_name	flight	aircraft	seat	class
Arjun Sharma	6E101	Airbus A320	12A	Economy
Arjun Sharma	6E404	Airbus A320	15C	Economy
Priya Nair	6E101	Airbus A320	14B	Economy
Rahul Kumar	AI202	Airbus A321	03C	Business

The query returns 8 rows; representative rows are shown.

3. Report 2 — booking count per flight

```
SELECT f.flight_number, COUNT(b.booking_id) AS total_bookings
FROM Flights f LEFT JOIN Bookings b
ON f.flight_id=b.flight_id
GROUP BY f.flight_id, f.flight_number
ORDER BY total_bookings DESC, f.flight_id;
```

flight_number	total_bookings
6E404	3
6E101	2
IX303	2
AI202	1

4. Report 3 — passengers booked on flight 6E101 using IN

```
SELECT passenger_id, full_name
FROM Passengers
WHERE passenger_id IN (
    SELECT passenger_id
    FROM Bookings
    WHERE flight_id=301
);
```

passenger_id	full_name
101	Arjun Sharma
102	Priya Nair

5. Report 4 — payments above average

```
SELECT payment_id, booking_id, amount
FROM Payments
WHERE amount > (SELECT AVG(amount) FROM Payments)
ORDER BY amount DESC;
```

payment_id	booking_id	amount
505	405	18000.00
503	403	12500.00
501	401	5500.00
507	407	5100.00

The average payment amount is 8250.00.

6. Report 5 — booking count by passenger

```
SELECT p.passenger_id, p.full_name,
       COUNT(b.booking_id) AS total_bookings
FROM Passengers p LEFT JOIN Bookings b
ON p.passenger_id=b.passenger_id
GROUP BY p.passenger_id, p.full_name
ORDER BY total_bookings DESC, p.passenger_id;
```

passenger_id	full_name	total_bookings
101	Arjun Sharma	2
102	Priya Nair	2
103	Rahul Kumar	1
104	Sneha Rao	1
105	Vikram Singh	1
106	Ananya Iyer	1

7. Report 6 — flights with no paid payment

```
SELECT f.flight_id, f.flight_number
FROM Flights f
WHERE NOT EXISTS (
    SELECT 1
    FROM Bookings b
    JOIN Payments pay ON b.booking_id=pay.booking_id
    WHERE b.flight_id=f.flight_id
      AND pay.payment_status='Paid'
);
```

With the sample data, no flight is returned because every flight has at least one paid booking.

8. Report 7 — booking summary by departure airport

```
SELECT f.departure_airport,
       COUNT(b.booking_id) AS total_bookings,
       COUNT(DISTINCT b.passenger_id) AS unique_passengers
FROM Flights f
JOIN Bookings b ON f.flight_id=b.flight_id
GROUP BY f.departure_airport
ORDER BY f.departure_airport;
```

departure_airport	total_bookings	unique_passengers
Bengaluru	5	4
Delhi	1	1
Mumbai	2	2

9. Report 8 — highest payment with passenger and flight

```
SELECT p.full_name, f.flight_number,
       pay.amount, pay.payment_status
FROM Payments pay
JOIN Bookings b ON pay.booking_id=b.booking_id
JOIN Passengers p ON b.passenger_id=p.passenger_id
JOIN Flights f ON b.flight_id=f.flight_id
WHERE pay.amount=(SELECT MAX(amount) FROM Payments);
```

passenger_name	flight_number	amount	status
Vikram Singh	6E404	18000.00	Paid

10. Conclusion

Joins combine related passenger, flight, booking, aircraft and payment information, while subqueries identify records using conditions derived from other queries. These reports support reservation management, payment monitoring and flight analysis.